Lab 3: Pipelined RISC-V Microarchitecture Implementation in SystemVerilog HDL

Instructor: Shahriar Shahabuddin
TA: Hridi Prova Debnath/Md Salman Sakib/KM Faisal

1. Introduction

In Lab 2, you implemented single-cycle RISC-V RV32I machine. In this lab, you will implement a pipelined version of that same microarchitecture. The pipeline must consist of five stages, as discussed in class, and covered in our textbook [1] and [2].

Now that multiple instructions can be "in-flight" at the same time, you must detect dependencies between instructions and handle them correctly. This lab's objective is to take Lab 2 and see the issues related to its pipelined implementation in hardware. For this laboratory, we will use the SystemVerilog Hardware Descriptive Language (HDL) modeling language. Hopefully, this laboratory will also allow you to see that executing things in parallel within hardware is not an easy choice despite being something the human body can do effortlessly. For full credit, you should achieve the following in this lab.

- Implement a pipelined RISC-V RV32I machine in SystemVerilog.
- Ensure that your implementation is correct by simulating it and verifying the register dumps of the test inputs.
- Implement your design on the National Instruments Elvis III and DSDB boards and the onboard SDRAM using the DDR3 built-in controller. And, compare your implementation in terms of execution with the single-cycle microarchitecture.

2. Specifications of the RISC-V RV32I Machine

2.1 Instruction Set Architecture

The machine should support all of the following RISC-V RV32I instructions, as with Lab 2

~~add~~	~~addi~~	~~andi~~	~~and~~	auipc
~~beq~~	bge	bgeu	blt	bltu
bne	jalr	~~jal~~	lb	lbu
lh	lhu	lui	~~lw~~	~~or~~
~~ori~~	sb	sh	sll	slli
~~slt~~	~~slti~~	sltiu	sltu	sra
srai	srl	srli	~~sub~~	~~sw~~
xor	xori			

2.2 Exceptions

No support (same as Lab 2).

2.3 Microarchitecture

Unlike a single-cycle microarchitecture, a pipelined microarchitecture divides the "work" required to execute an instruction across multiple cycles. Each cycle corresponds to a stage within a pipeline. The major advantage of a pipelined microarchitecture is that it can execute multiple instructions in parallel: multiple instructions can be in the pipeline at the same time, albeit at different stages.

However, the major difference is the control cannot be centralized like the single-cycle or multi-cycle architectures. They have to be passed down the pipelined in order to simplify its complexity. In order to help alleviate problems between stages, a Hazard Detection Unit (HDU) (through the `hazard` SystemVerilog module), as mentioned in class, is utilized to keep things working without conflicts.

2.3.1 Pipeline Stages

For this lab, you must implement the following five-stage pipeline. Please ensure that there are exactly five stages. Please ensure that each stage does exactly what it is supposed to do (no more, no less). For example, as long as the memory is accessed only during the MEM stage, you are allowed to generate control signals (or perform other bookkeeping activities) for other stages. Later on, for extra credit, you will be allowed to design your own custom pipeline if you want.

Stage	Name	Specification
1	IF	Instruction Fetch
2	ID	Instruction decode and register file read
3	EX	Execution or memory address calculation
4	MEM	Memory access
5	WB	Writeback to the register file

2.3.2 Handling Data Dependence

For this lab, you must implement a five-stage pipelines that handles data dependences. It should handle the following methods for data dependence as discussed in your textbook and class [1]

1. Stalling. When a data dependence is detected, simply prevent later instructions from entering/progressing through the pipeline. This leads to idle pipeline stages referred to as "bubbles". For this lab, your implementation must stall only when necessary.

2. Forwarding. When a data dependence is detected, allow an earlier instruction to send data directly to a later instruction even before the data has been written back into the register file. For this lab, you must forward data into the end of the decode stage. Your implementation is still allowed to stall, but only when stalling cannot be prevented by forwarding data.

2.4 Memory

As mentioned in class, the previous lab and in the textbook, the memory is byte addressable. As previously in lab 2, the memory module handles the memory as a Harvard architecture. That is, there is one memory where the instructions are stored and the other where data is stored.

Just as we learned in class with PC+4 being a historical item, many companies use a specific endianess because of historical reasons. The true RISC-V RV32I architecture has the ability to switch endianess with a switch at boot. If you run the sample program (i.e., `riscvtest.S`) correctly, as stated in the assembly file, it should write the value of 7 to address 100. You can check the result of other programs by using the Spike simulator.

3. Synthesis and Implementation on the National Instruments ELVIS III board

For this lab as with the previous lab, we will also implement our architecture into a real device with real memory. The board we plan on using is the National Instruments Virtual Implementation Suite or NI ELVIS III board. In addition to the ELVIS III board, we will attach an add-on device to give us use of a Xilinx Zynq-7Z020 FPGA. For this, we will be using the National Instruments Digital System Development Board (DSDB). The DSDB board is also populated with two Micron DDR3 memory components to give a single

rank, 32-bit wide interface with a total of 512 MB of data memory. More information on DRAM and the DDR interface can be found in [3].

As with the single-cycle microarchitecture, you can use the sources panel to add the `.memfile` file of your choice to the project, then make sure the `$readmemh` line in `imem.v` is correctly referencing the name of the file you wish to use. The `.memfile` file actually stores the instruction memory on the FPGA through use of Block RAM or BRAM that AMD FPGAs have. BRAM is significantly smaller than our external DRAM memory, however, it will be close to the processor and, subsequently, fast in terms of access. If you forget to including your `.memfile` file, Vivado may lock up during synthesis trying to find a file that hasn't been added to the project. If this happens, you will have to cancel the synthesis and then add the file. Consult the `fpga_guide` document on Canvas for more information on this process.

Again, similar to lab 2, to make sure the data gets to the processor correctly, a memory controller is used. Most designs today use a fast memory controlled called a Double Data Rate (DDR) synchronous interface. This memory controller is crucial to making the Synchronous DRAM works correctly. It speeds up the transfer by utilizing both edges of the clock. Fortunately, there is a DDR3 controller on the FPGA, so we will use this IP that is already there through the AXI interface via a memory mapped address. The Double Data Rate (DDR) memory controller consists of three major modules: a core memory controller and scheduler (DDRC), an AXI memory port interface (DDRI) and a digital PHY and controller (DDRP) [4]. Although the DDR controller allows the transfer of data to/from the processor easily, it is an asynchronous protocol in that it has to wait for the memory to respond that it is complete.

4. Synthesis and Area Comparison Using ASIC Tools

For this lab, as with the previous lab, we will implement and compare our architecture using synthesis with Sky90 technology. Please use the server port provided for students on Canvas. Navigate to the **synthDC/synth** repository. In the **hdl** folder, save your RISC-V module. At line 27, change the module name in the **synthDC/synth/scripts/synth.tcl** file. Then, follow the steps below:

1. **Navigate to the synthesis directory:**

   ```
   cd synthDC/synth/
   ```

2. **Launch the required shell environment:**

   ```
   tcsh
   ```

3. **Clean intermediate and output files:**

   ```
   make clean
   ```

4. **Run synthesis:**

   ```
   make synth
   ```

5. **Review reports and compare with the single-cycle processor:**

   After a successful synthesis, the area and timing reports will be located in the **synthDC/synth/reports/** folder. A successful result indicates that your design ran as expected, which is essential for validating the correctness of the instruction execution flow.

   Open the timing report and check whether your design's critical path is **MET** or **NOT MET**. Next, adjust the `clk_freq` parameter in **synth.tcl** to determine the frequency range over which timing is met. Finally, compare your pipelined implementation (`riscv_pipelined`) against your single-cycle implementation from Lab 2 (`riscv_single`) in terms of both **area** and **timing (MET range)**.

5. Lab Resources

The source code for the lab is provided through the canvas as with previous labs. The source code is written in SystemVerilog, a very popular hardware description language. If you need a refresher on SystemVerilog, please refer to Chapter 4 of the Harris/Harris text. There is copious information on the architecture in Chapter 7 in the DDCA text [2] as well. Our textbook also has some great information on this microarchitecture too in Chapters 2 and 4 [1]. As mentioned on the documents within Canvas, making your SystemVerilog resemble the digital logic you want to implement will make things easier. Do not be tempted to Google code or implement something behaviorally (i.e., caveat emptor).

6. Getting Started & Tips

1. In the top-level skeleton (`riscv_pipelined.sv`), it already implements 13 instructions:

   ```
   add, addi, and, andi, beq, jal, lw, or, ori, slt, slti, sub, sw
   ```

   You should use these to understand how the processor works and modify the datapath/control to add the remaining instructions. Please note that LW/SW <u>only</u> implements offset-based load and store instructions. Preindex and postindex types of instructions can be implemented, but might require a considerable hardware investment and/or additional cycles.

2. Since it takes additional cycles to commit an operation in addition to the complexity of the pipelined RISC-V RV32I microarchitecture, I highly recommend starting this lab early. Good use of your DO file to highlight areas of the pipeline may also be helpful.

3. Review the material on pipelined machines from the lecture notes and the DDCA textbook [2]. The whole key to pipelining is to make sure that all signals you want to pass to the next stage go through a storage device. If you forget to store a signal, you will have an error sometime in your processor's lifetime. Some signals bypass the registers, but have to be maintained by the control logic which complicates operation for concurrency.

4. Do not modify the `imem` or `dmem` modules other than the number of elements in the `RAM[63:0]` definition within the `imem` module to expand the size of your program. That is, this definition indicates 64 locations for your program that is read in through the testbench (i.e., for a given `.memfile`). If you have a larger program, such as the one in the testing subdirectory, you will need to make this size larger.

5. At this point, you should be able to simulate the test input `riscvtest.memfile` by executing the following command in your shell: `vsim -do riscv_pipelined.do`

6. You should check for correctness by comparing the register dump of the simulation against a reference register dump (e.g, through your simulation experience from Lab 1). Recall that this takes time for your design to complete the operation as each instruction has to pass down the pipeline, as opposed to a single-cycle version.

7. It is far easier to add instructions one at a time and check to make sure they work before going on to the next instruction.

8. As with lab 2, it is easier to implement datapath functionality first within the datapath module and then make sure it works by completing the control module afterwards.

9. You should also analyze the performance of the `test_hw.S` in your repository with the ELVIS III board. Describe whether memory affected or did not affect your design. Also, compare your results with the single-cycle microrarchitecture and determine which one is faster. Explain your reasoning within your lab report.

6.1 Tips

- If you have not had me before in a class, you are pretty much not going to finish this or any assignment if you start late - guaranteed[1]! Do not believe me, ask others who have had my classes. So, start early and apportion your time every day, if possible.
- Read this handout closely. Ask questions to the TA or me using slack or in person.
- It is really easy to start with R-type instructions (e.g., `xori`) as with lab 2.
- The smaller load and stores (e.g., `lb`) logic should be in the datapath and not somewhere else.
- When you encounter a technical problem, please sift through your logs file and look any error specifically. Many times the error tells you the problem.
- Be cautious about how SystemVerilog handles signed/unsigned numbers.
- I have modified the DO file to segment the waveform to order the screen so it is easier to read as with lab 2.
- Memory accesses <u>must</u> be aligned. If you are accessing a byte, any address is allowed. But, if you are accessing a 4-byte word, the two lowest bits of the address must be zero.
- The cycle time of your implementation is determined by the critical path of the slowest stage. While we do not really check the cycle time, mainly due to not synthesizing your design, you should still try to keep your cycle time as short as possible. Keep in mind that cycle time is not the whole story. Forwarding paths may actually increase your cycle time, but also improve overall performance (by reducing stalls and decreasing CPI).
- Finally, it is important to make your processor work first, and then make it work fast. Premature optimization is the root of all evil.

7. Handin

You should electronically hand in your code into Canvas as with Lab 2. Please contact the TAs or the instructor for more help. Your code should be readable and well-documented. In addition, please turn in additional test cases that you used in a inputs/ subdirectory. If you feel the need to describe any additional aspects of your design in detail, please include these in a separate README. Make sure that you test your SystemVerilog extensively with a suite of test cases, just like in Lab 1, so that you are confident that you have implemented all instructions correctly.

To get the maximum grade for this lab, your SystemVerilog must be able to simulate all the instructions asked in Section 2. Your job is to really try to make sure your simulator covers all possible instructions that this unknown code may utilize.

8. Extra Credit

Again, there are many possibilities for extra credit, but you should only attempt this if you get the baseline project done and you have extra time.

References

[1] D. Harris, J. Stine, S. Harris, and R. Thompson, *RISC-V Microprocessor System-On-Chip Design*. Elsevier Science, 2025.

[1] Luck favors the prepared!

[2] S. Harris and D. Harris, *Digital Design and Computer Architecture, RISC-V Edition*. Elsevier Science, 2021.

[3] B. Jacob, S. Ng, and D. Wang, *Memory Systems: Cache, DRAM, Disk*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2007.

[4] L. H. Crockett, R. A. Elliot, M. A. Enderwitz, and R. W. Stewart, *The Zynq Book: Embedded Processing with the Arm Cortex-A9 on the Xilinx Zynq-7000 All Programmable Soc.* UK: Strathclyde Academic Media, 2014.